

Inclusión local de ficheros (LFI)

Objetivos de aprendizaje

Distinguir selección de ruta, lectura e inclusión; reconstruir base y sufijo; identificar el componente que interpreta el fichero; adaptar profundidad, encoding o wrapper solo con evidencia.

Prerrequisitos

Rutas relativas y canónicas, percent-encoding, permisos Linux, HTTP y método de hipótesis.

Fundamentos técnicos

Path traversal es la capacidad de salir de una ubicación prevista mediante una ruta controlada. Lectura arbitraria describe el efecto de un sink como `readfile()` u `open()`. LFI implica que una aplicación incluye un fichero local mediante un mecanismo como `include`; en PHP, `include` incluye y evalúa el fichero, pero el código ejecutable debe aparecer en etiquetas PHP válidas. Por tanto:

control de ruta != lectura garantizada != `include` != ejecución controlada

Cada flecha requiere precondiciones y evidencia propias.

Modelo mental

entrada -> decode -> prefijo + valor + sufijo -> normalización -> permisos
-> open/read/include -> bytes servidos o interpretación -> respuesta

Superficie de ataque

Parámetros de página, idioma, tema, plantilla, documento o descarga. También rutas almacenadas en base de datos y cabeceras que seleccionan localización. El nombre `file` no revela el sink.

Cómo identificarla

- Un error muestra la ruta construida, prefijo o sufijo.
- Un fichero marcador conocido produce contenido inequívoco y un control inexistente produce un error distinto.
- En un `include`, texto fuera de etiquetas puede mostrarse mientras código válido se evalúa.
- Un wrapper solo es relevante si el runtime, la función y la configuración lo soportan.

Preguntas que debo hacerme

1. ¿Cuál es la base y cuántos niveles necesito resolver?
2. ¿La aplicación añade extensión o prefijo?
3. ¿El sink lee bytes o incluye/evalúa?
4. ¿Qué identidad y permisos tiene el proceso?
5. ¿Cuántas decodificaciones y normalizaciones ocurren?

Prueba mínima

Usa un fichero marcador del laboratorio con contenido conocido y un hermano inexistente. No empieces por credenciales ni por ejecución. Un error con ruta absoluta es una pista sobre construcción; no confirma lectura.

Construcción progresiva del payload

Escenario:

```
include __DIR__ . '/views/' . $_GET['page'] . '.php';
```

Entrada normal home produce `/srv/app/views/home.php`. Para alcanzar `/srv/app/markers/probe.php`, la ruta debe subir un nivel desde `views`, entrar en `markers` y tener en cuenta que `.php` ya se añade. La construcción es

../markers/probe, no una cantidad arbitraria de ../.

Si el error revela otra base, se recalcula. Si el proceso no puede leer el fichero, aumentar profundidad no adapta el problema.

Anatomía de los payloads

- Contexto de entrada: query page.
- Sintaxis original: `include '/srv/app/views/' + page + '.php'`.
- Entrada controlada: segmento central.
- Transformaciones conocidas: URL decode una vez.
- Parser final: resolución de ruta y `include` de PHP.
- Sink: inclusión/evaluación local.
- Primitiva: incluir un marcador PHP conocido fuera de views.
- Payload mínimo: `../markers/probe`.
- Significado de cada componente: `..` selecciona `/srv/app`; `markers/probe` selecciona objetivo; el sufijo completa `.php`.
- Resultado esperado: marcador emitido por el fichero.
- Control negativo: `../markers/missing`.
- Restricción observada: el sufijo `.php` se añade.
- Por qué falla la variante básica: `../../etc/passwd` se convierte en `/srv/etc/passwd.php`, objetivo y sufijo incorrectos.
- Hipótesis de adaptación: usar un objetivo compatible con el sufijo o demostrar un mecanismo que transforme la lectura.
- Payload adaptado: `../markers/probe`.
- Por qué debería funcionar: la ruta normalizada existe y es legible.
- Evidencia: salida única del marcador y error diferente para el control.

Cuándo no funcionaría: allowlist de vistas, canonicalización dentro de base, permisos insuficientes o sink de lectura distinto.

Variaciones según el contexto

Sink	Resultado mínimo	¿Interpreta?
<code>readfile(path)</code>	bytes del fichero	no por sí mismo
<code>open(path)</code> + respuesta	bytes si la app los devuelve	no
<code>include(path)</code> PHP	salida y evaluación de PHP válido	sí, según contenido
servidor estático	recurso bajo raíz autorizada	depende del handler, no del nombre

`php://filter/convert.base64-encode/resource=...` puede transformar un recurso en PHP cuando el wrapper y el filtro están disponibles. No elimina automáticamente un sufijo añadido ni existe en otros runtimes.

Filtros y bypasses

Clasifica el control: rechazo textual de `../`, normalización canónica, allowlist de identificadores, comprobación de prefijo final o permisos. Solo una validación antes de una decodificación posterior justifica probar una representación codificada equivalente.

Evidencias de confirmación

Contenido inequívoco del fichero o salida de un marcador incluido, con control inexistente. Un warning confirma que se intentó resolver una ruta; no que el objetivo se leyó ni que su contenido se interpretó.

Escalado de impacto

Determinar base, sufijo y sink; confirmar un marcador; mapear permisos; leer solo ficheros necesarios del laboratorio; estudiar interpretación únicamente si se controla contenido de un fichero que el sink incluye.

Errores frecuentes

- Llamar LFI a toda descarga con nombre controlado.
- Suponer que muchos `../` siempre ayudan.
- Ignorar el sufijo añadido.
- Usar wrappers sin comprobar runtime y configuración.
- Confundir warning, lectura y ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Experimento discriminatorio
error termina en <code>.php</code>	sufijo añadido	objetivo existente con y sin extensión en entrada
ruta absoluta ignora el prefijo	función acepta ruta absoluta	comparar marcador absoluto y relativo autorizado
<code>../</code> se elimina una vez	filtro textual no recursivo	observar valor posfiltro; no probar variantes a ciegas
objetivo existe pero <code>Permission denied</code>	identidad sin lectura	comprobar UID y permisos, no aumentar profundidad
fuelle PHP no aparece	<code>include</code> lo evalúa	marcador de texto y fichero con salida conocida
wrapper se imprime o falla como esquema	sink/runtime no lo reconoce	confirmar PHP, función y wrappers habilitados

Mitigaciones

Mapa fijo de identificadores a plantillas, rutas generadas por la aplicación, canonicalización y comprobación de pertenencia a la base, privilegio mínimo y errores no verbosos. Validar extensión sin comprobar la ruta final no resuelve traversal.

Relación con pentesting y certificaciones

Se espera justificar la ruta normalizada y el sink. El éxito no se mide por alcanzar `/etc/passwd`, sino por explicar qué primitiva se confirmó.

Caso guiado

Caso A — Básico: una vista fuera del directorio

`page=home` carga `/opt/site/views/home.php`. `page=missing` muestra `include(/opt/site/views/missing.php)`. Existe un marcador de laboratorio en `/opt/site/markers/probe.php`.

Observación: el error revela base y sufijo. Qué sé: se construye una ruta de `include`. Hipótesis: el valor admite segmentos relativos o se aplica `basename`. Experimento: `../markers/probe` y control `../markers/missing`.

Predicción: si se resuelve relativo, el primero emite el marcador; con basename, ambos quedan en views. Resultado: aparece INCLUDE_OK; el control genera warning. Conclusión: inclusión local fuera de la base. Siguiendo paso: registrar permisos y detenerse antes de buscar ejecución adicional.

Caso de adaptación

Caso B — El objetivo correcto recibe un sufijo

`../../../../etc/passwd` produce warning sobre `/opt/etc/passwd.php`.

Observación: profundidad y sufijo no coinciden con la hipótesis. Qué sé: `.php` se añade y la base está en `/opt/site/views`. Hipótesis: no puede leerse ese objetivo con esta construcción; quizá existe un wrapper compatible o un fichero `.php` útil del laboratorio. Experimento: primero un marcador `.php` conocido; después, solo si PHP y wrappers están confirmados, comprobar si el wrapper llega intacto al sink. Resultado: el marcador funciona; el wrapper es rechazado por allowlist de esquema. Conclusión: se confirma LFI de `.php`, no lectura arbitraria de cualquier fichero. Siguiendo paso: delimitar impacto real.

Caso C — Transferencia: selector de idioma

Una cookie `lang=es` carga textos. Al enviar `lang=../../../../markers/probe`, aparece INCLUDE_OK; el título del ejercicio no revela la técnica.

Resolución: cookie -> concatenación `/translations/` -> include -> fichero. El mismo control de ruta aparece fuera de `page=`. Un control inexistente separa inclusión de contenido cacheado.

Caso D — Falso positivo: error de plantilla simulado

`page=../../../../x` devuelve una página que imprime literalmente `failed to open stream: /views/../../../../x.php`, pero el mismo texto aparece en el JavaScript del frontend y no cambia con rutas distintas.

Observación: mensaje compatible con PHP. Hipótesis: warning backend o simulación client-side. Experimento: repetir sin JavaScript, cambiar marcador único y revisar respuesta HTTP cruda. Resultado: el servidor siempre devuelve el mismo JSON y el navegador compone el texto. Conclusión: no hay evidencia de filesystem ni include. Siguiendo paso: cerrar la hipótesis.

Ejercicios

1. Normaliza `/srv/app/views/../../../../markers/a.php` y explica cada segmento.
2. Diseña un control que distinga `readfile` de `include` usando dos marcadores seguros.
3. Un wrapper falla con "scheme not allowed": enumera tres causas y una prueba para cada una.
4. Explica por qué `Permission denied` confirma una ruta intentada, no lectura.

Resumen

LFI exige identificar construcción de ruta y sink. Traversal, lectura, include e interpretación son capacidades distintas; cada una se confirma por separado.

Chuleta operativa

1. Base y working directory.
2. Prefijo y sufijo.
3. Decodificaciones.
4. Ruta normalizada.
5. Identidad y permisos.
6. Read frente a include.
7. Marcador y control inexistente.
8. Wrapper solo con evidencia.

Referencias

- [PortSwigger: path traversal](#)
- [PHP: include](#)
- [PHP: wrappers](#)
- [Concepto interno](#) (1-f1)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).